

PROYECTO FINAL

Sistema de Orquestación de Soporte Técnico



UNIVERSIDAD DE ALMERÍA

Integración de Tecnologías y Servicios Informáticos

Juan Carlos Díaz González – Isabel Pelaya Galindo Ibáñez –
Esther Ibáñez Mingorance – José Luis López García

Índice

1.	Descripción del Flujo de Trabajo.....	2
1.1.	Disparador: Recepción de Soportes (Nuevo Componente).....	2
1.2.	Inteligencia Artificial: Triage y Extracción	3
1.3.	Persistencia y Enriquecimiento.....	4
1.4.	Lógica de Enrutamiento y Desacoplamiento	6
1.5.	Modularidad y Notificación	8
2.	Flujo de Trabajo Secundario: Manejo de Errores	10
2.1.	Activación del flujo mediante Error Trigger	10
2.2.	Registro persistente de errores	11
2.3.	Notificación automática por correo electrónico	12
3.	Flujos resultantes.....	13
4.	Establecer las variables de entorno.....	14
4.1.	Variables de entorno utilizadas	14
4.1.1.	Base de datos – Postgre SQL.....	14
4.1.2.	Cola de mensaje – RabbitMQ.....	15
4.1.3.	N8N.....	15
4.1.4.	Cuenta de correo – Notificaciones	16
4.1.5.	Registro de errores – Google Sheet.....	16
4.2.	Ejemplo de .env.example.....	16
4.3.	Establecer las variables de entorno en n8n.....	17

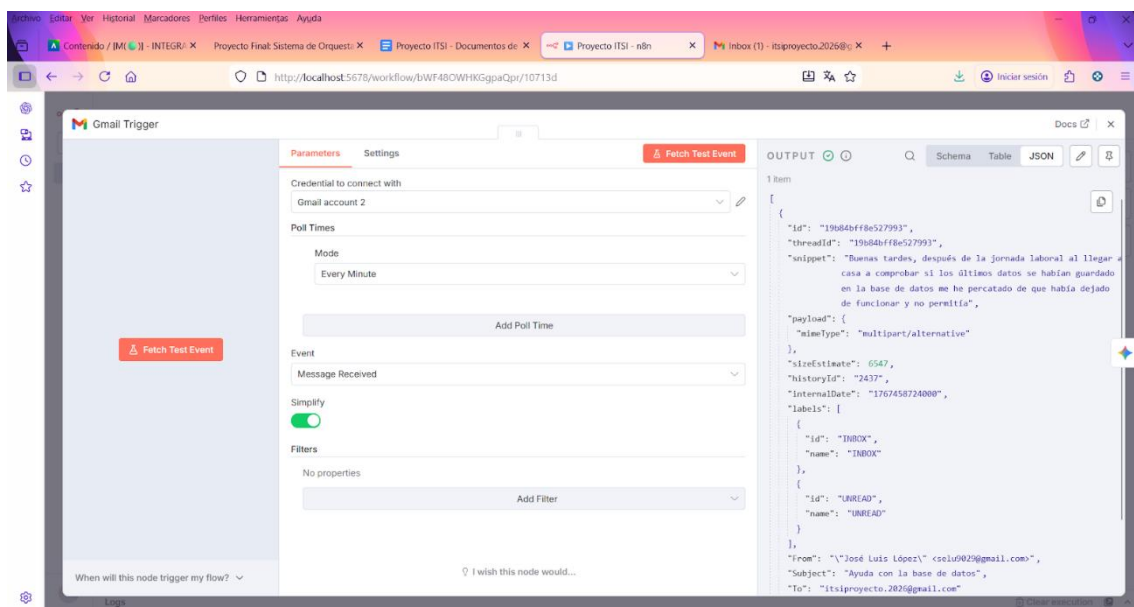
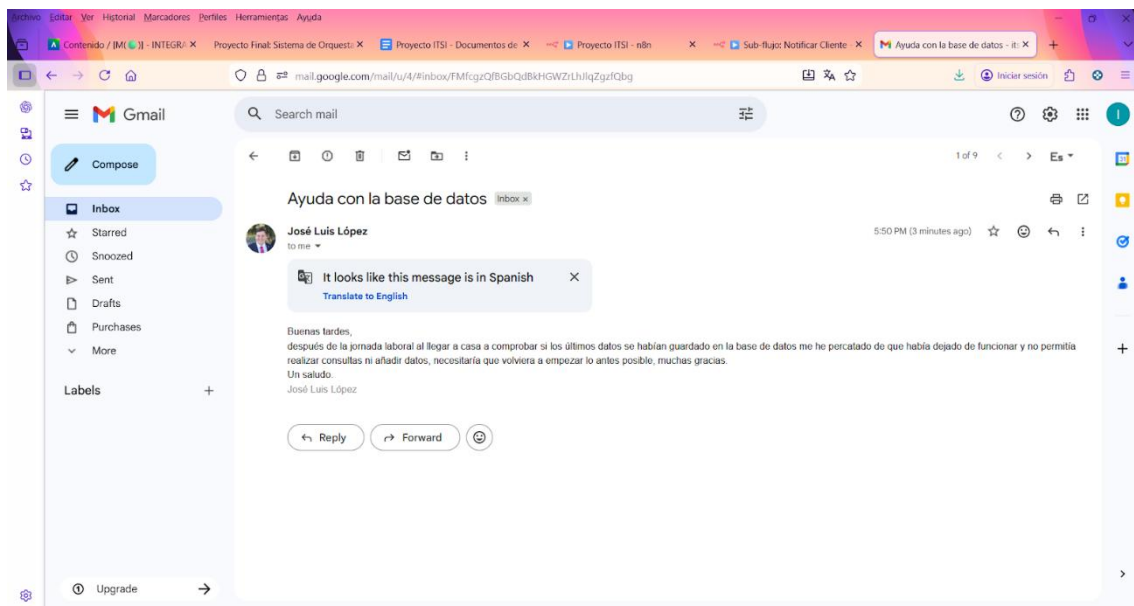
1. Descripción del Flujo de Trabajo

Este flujo representa el proceso completo de ingesta y triaje de un nuevo ticket de soporte.

1.1. Disparador: Recepción de Soportes (Nuevo Componente)

El flujo de trabajo se activa automáticamente cuando se recibe un nuevo correo electrónico en una cuenta de soporte dedicada, utilizando para ello el nodo Gmail Trigger de n8n.

El objetivo es conectar n8n a una cuenta de correo electrónico dedicada. El trigger se activará cuando llegue un nuevo mensaje, extrayendo la información básica del correo, concretamente el emisor y el cuerpo.

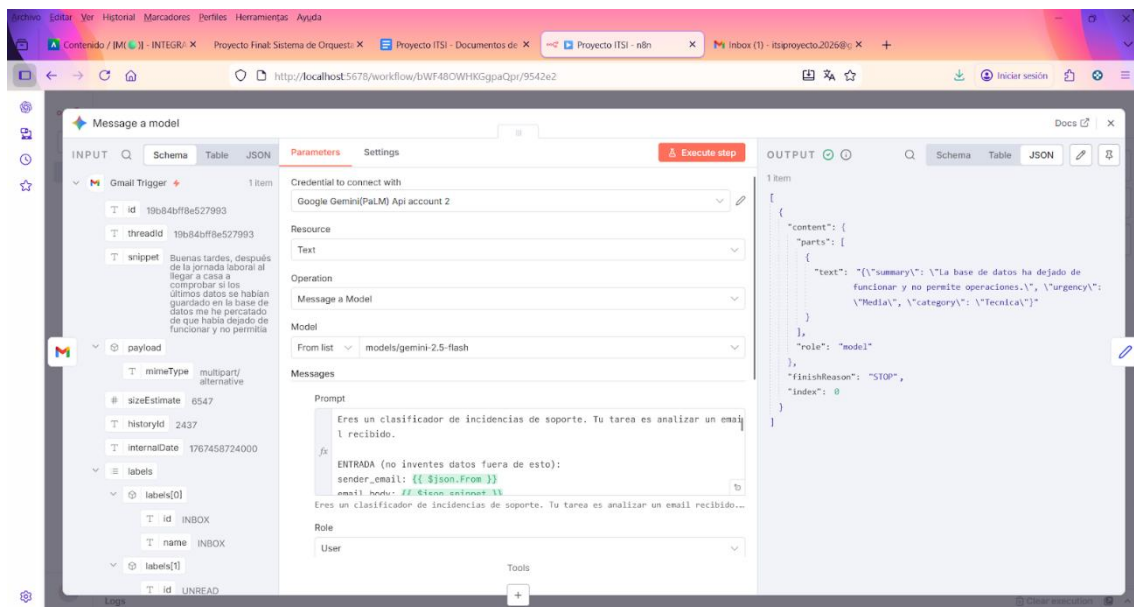


1.2. Inteligencia Artificial: Triage y Extracción

Una vez recibida la incidencia por correo electrónico, el siguiente paso consiste en analizar el contenido del mensaje, que se presenta en formato no estructurado, con el objetivo de transformarlo en información estructurada que pueda ser utilizada por el resto del sistema.

Para ello, se emplea un nodo de Inteligencia Artificial basado en el modelo Gemini. Diseñamos un prompt que recibe como entrada el correo electrónico del emisor y el cuerpo del mensaje. Como salida, devolverá únicamente un objeto JSON válido que contenga:

- **summary:** Un resumen de una frase del problema.
- **urgency:** ("Alta", "Media", "Baja").
- **category:** ("Tecnica", "Facturacion", "Ventas", "General").



El *prompt* introducido es:

Eres un clasificador de incidencias de soporte. Tu tarea es analizar un email recibido.

ENTRADA (no inventes datos fuera de esto):

sender_email: {{ \$json.From }}

email_body: {{ \$json.snippet }}

INSTRUCCIONES ESTRUCTURADAS:

1) Debes responder ÚNICAMENTE con un objeto JSON válido (sin texto adicional).

2) No uses Markdown, no uses comillas invertidas, no uses explicaciones.

- 3) El JSON debe contener EXACTAMENTE estas 3 claves: "summary", "urgency", "category".
- 4) "summary" debe ser una sola frase corta (máximo 20 palabras) que describa el problema principal.
- 5) "urgency" debe ser exactamente una de: "Alta", "Media", "Baja".
- 6) "category" debe ser exactamente una de: "Tecnica", "Facturacion", "Ventas", "General".
- 7) Si el email no aporta información suficiente, usa:

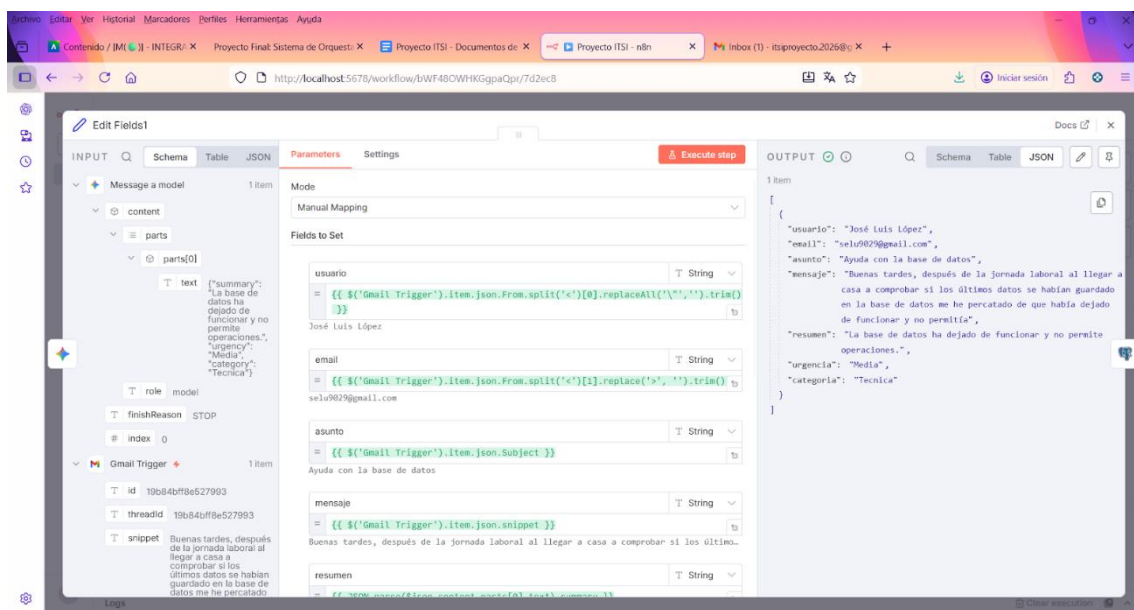
summary: "Solicitud poco clara; se requiere más información."

urgency: "Baja"

category: "General"

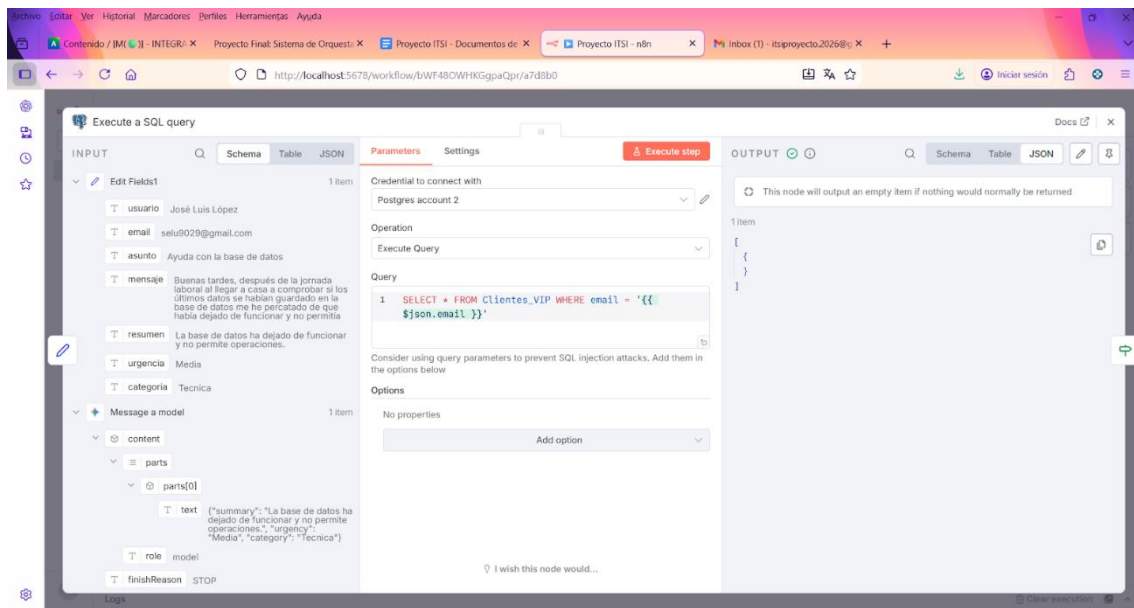
- 8) No incluyas saltos de línea dentro de los valores (mantén strings simples).

Una vez que Gemini lo ha clasificado, añadimos un nodo Edit Fields para adaptar todos los campos y que quede bien categorizado:

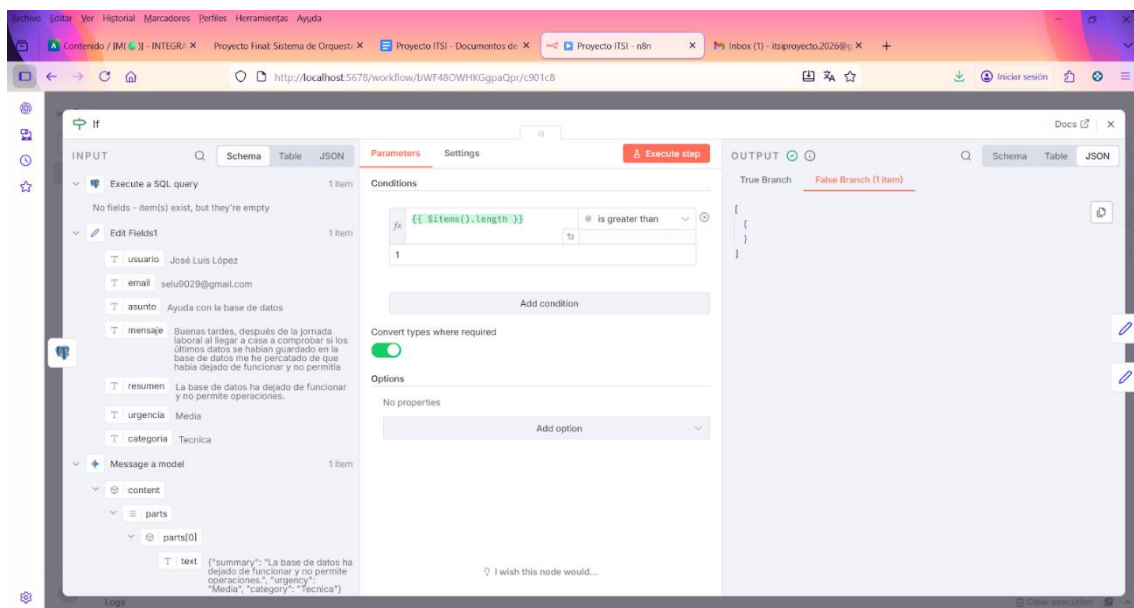


1.3. Persistencia y Enrichimiento

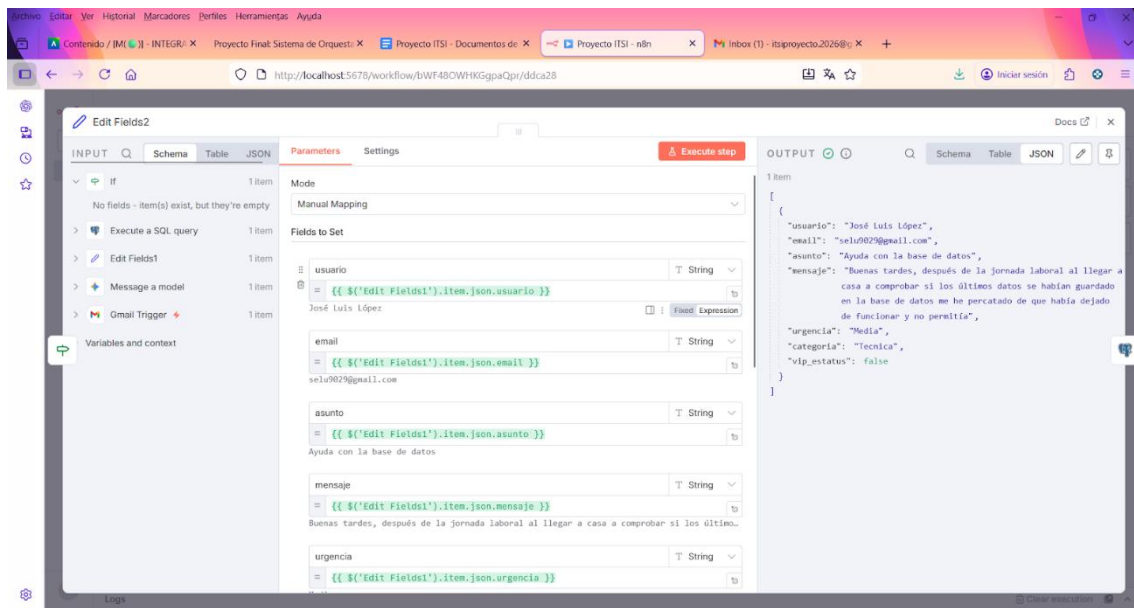
Una vez obtenidos los datos estructurados combinados con la información original del correo electrónico, interactuamos con la base de datos para consultar si el cliente es conocido y si tiene estado VIP.



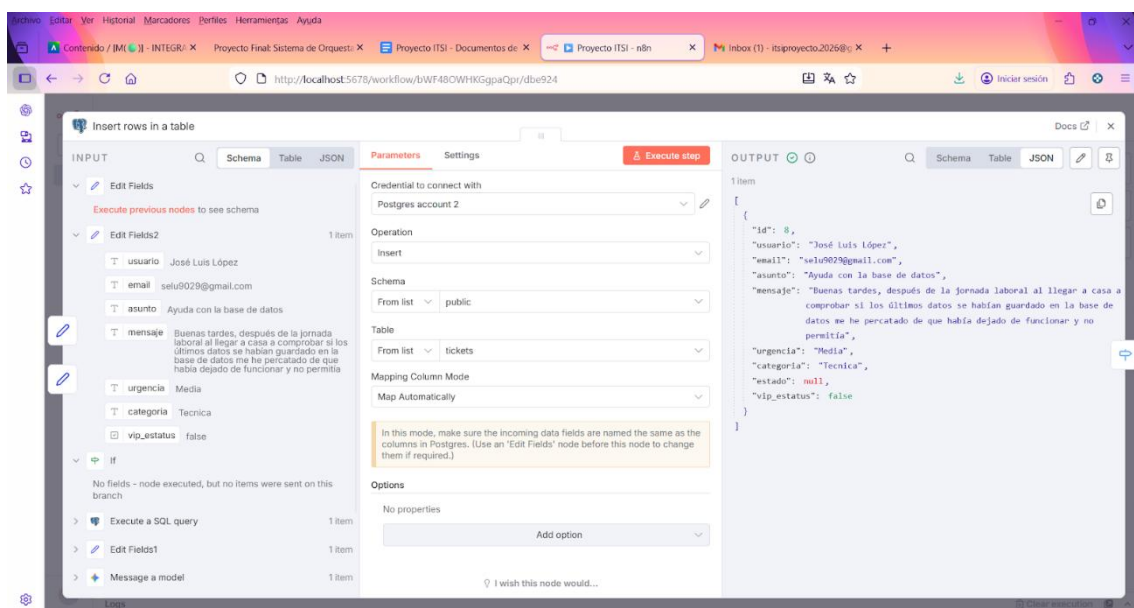
A continuación, el resultado de esta consulta se evalúa mediante un nodo if. En base a este nodo, el flujo distingue entre clientes conocidos y clientes no registrados, estableciendo el valor del campo vip_estatus como verdadero o falso:



Además, seguiremos conservando aquellos campos que siguen siendo relevantes en el proceso, para posteriormente insertarlos en base de datos.



Finalmente, insertamos un nuevo ticket en la base de datos mediante un nodo PostgreSQL:



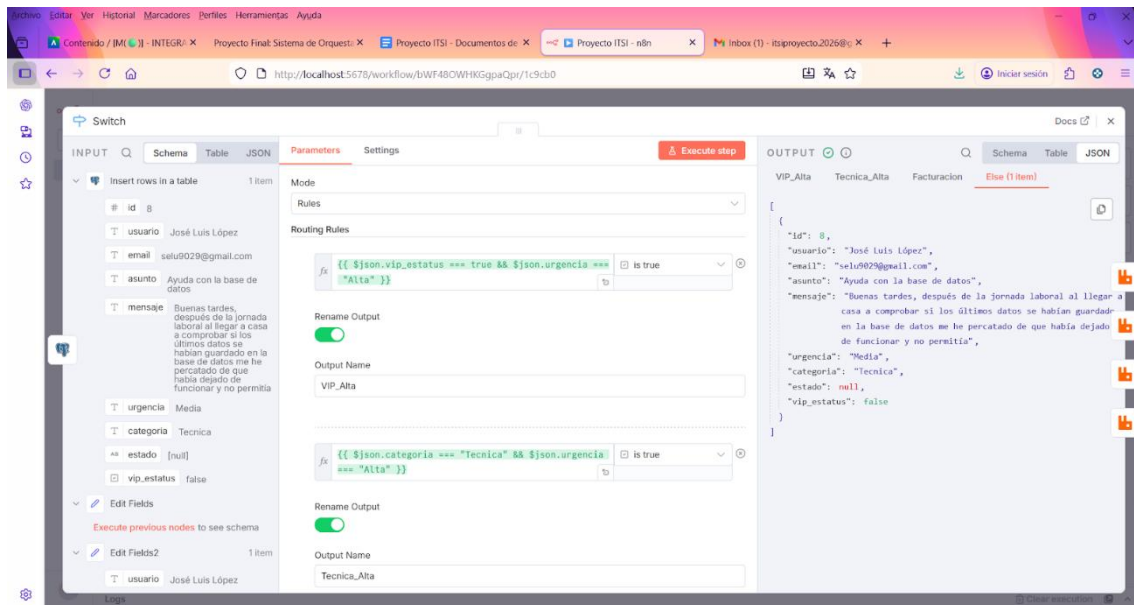
1.4. Lógica de Enrutamiento y Desacoplamiento

Basándose en todos los datos recopilados (IA + BBDD), el flujo debe decidir qué hacer con el ticket, usando un nodo Switch.

El nodo debe enrutar el ticket basándose en una combinación de campos:

- **Caso 1:** `{{ $json.vip_status }}` es true Y `{{ $json.urgency }}` es "Alta".

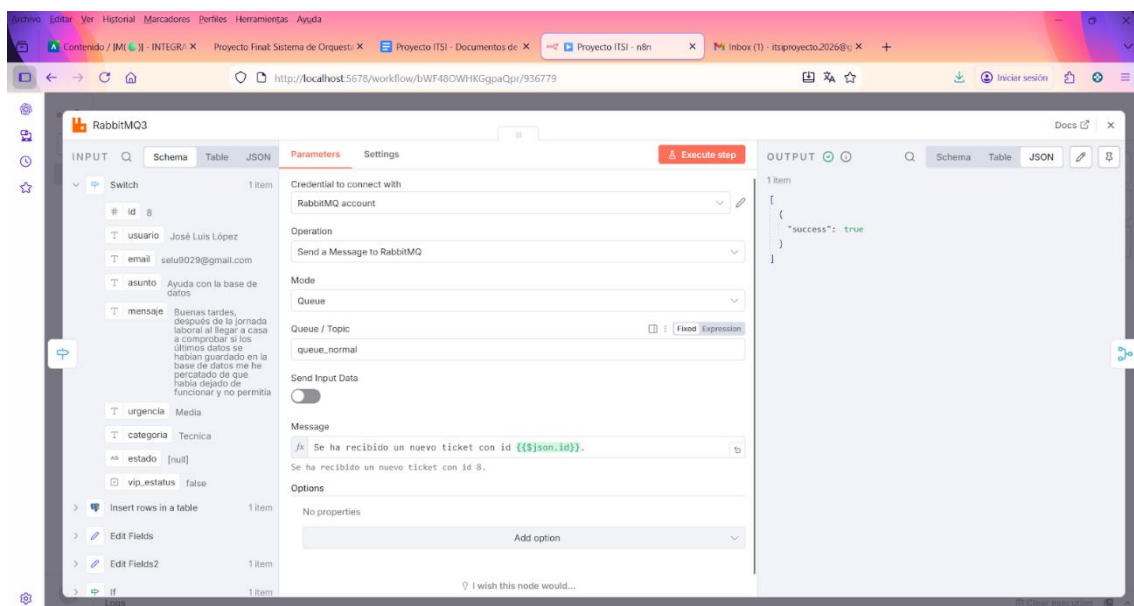
- **Caso 2:** `{{ $json.category }}` es "Tecnica" Y `{{ $json.urgency }}` es "Alta".
- **Caso 3:** `{{ $json.category }}` es "Facturacion".
- **Caso 4:** Resto de los casos.



Cada una de las ramas del nodo Switch se conecta a un nodo RabbitMQ, encargado de enviar un mensaje a una cola diferente según el tipo de ticket. Las colas utilizadas son:

- `queue_vip_urgent`
- `queue_technical_urgent`
- `queue_billing`
- `queue_normal`

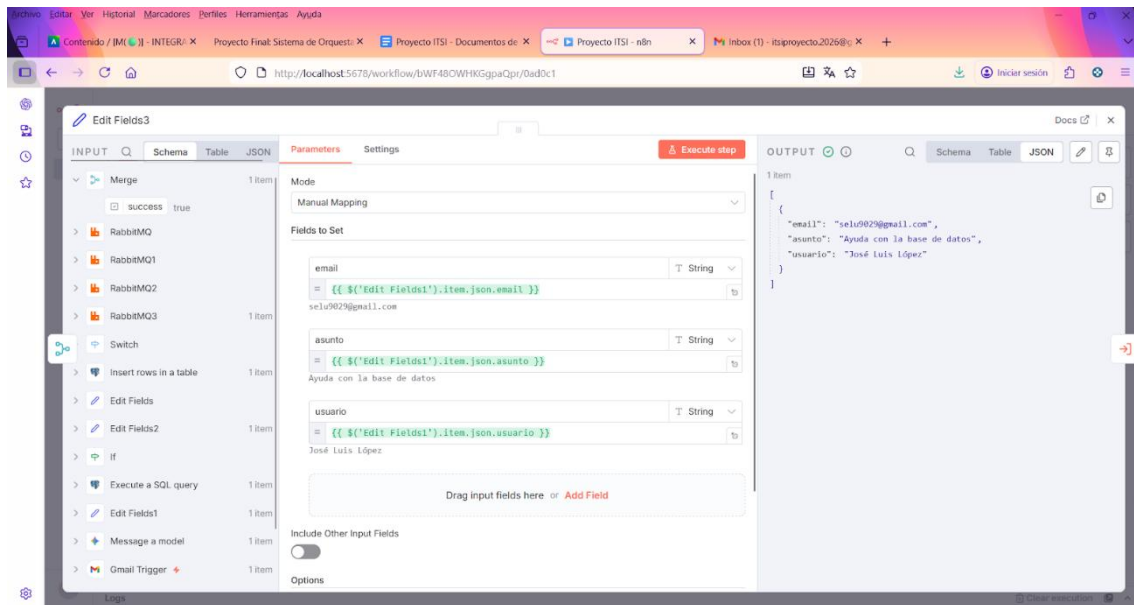
El mensaje enviado a RabbitMQ contiene el ID del ticket de la base de datos.



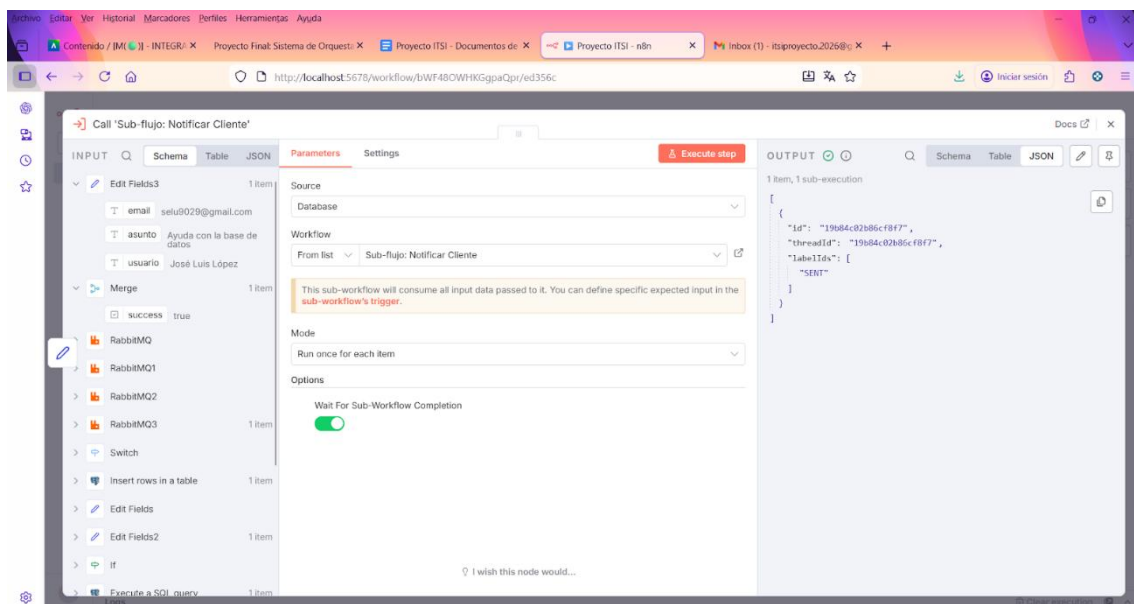
1.5. Modularidad y Notificación

Independientemente del enrutamiento, se debe notificar al cliente que su ticket ha sido recibido. Crearemos un sub-flujo "Sub-flujo: Notificar Cliente" que comience con un Execute Workflow Trigger.

Antes de hacer la llamada al sub-flujo, añadimos un nodo Edit Fields para preparar los campos que se le van a mandar a Notificar Cliente.

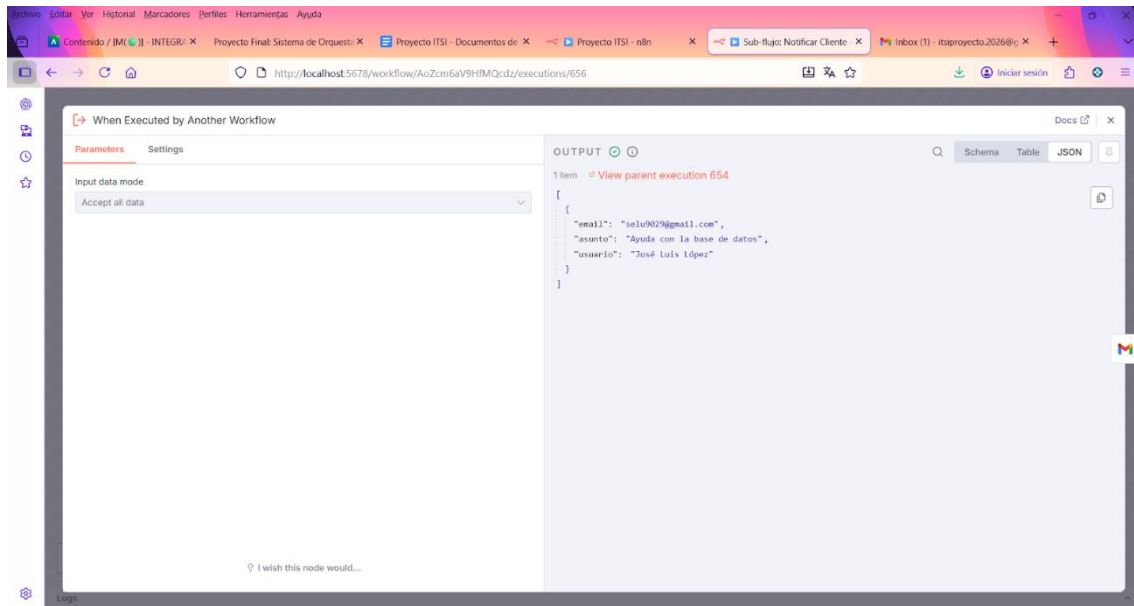


Hacemos la llamada al sub-flujo.

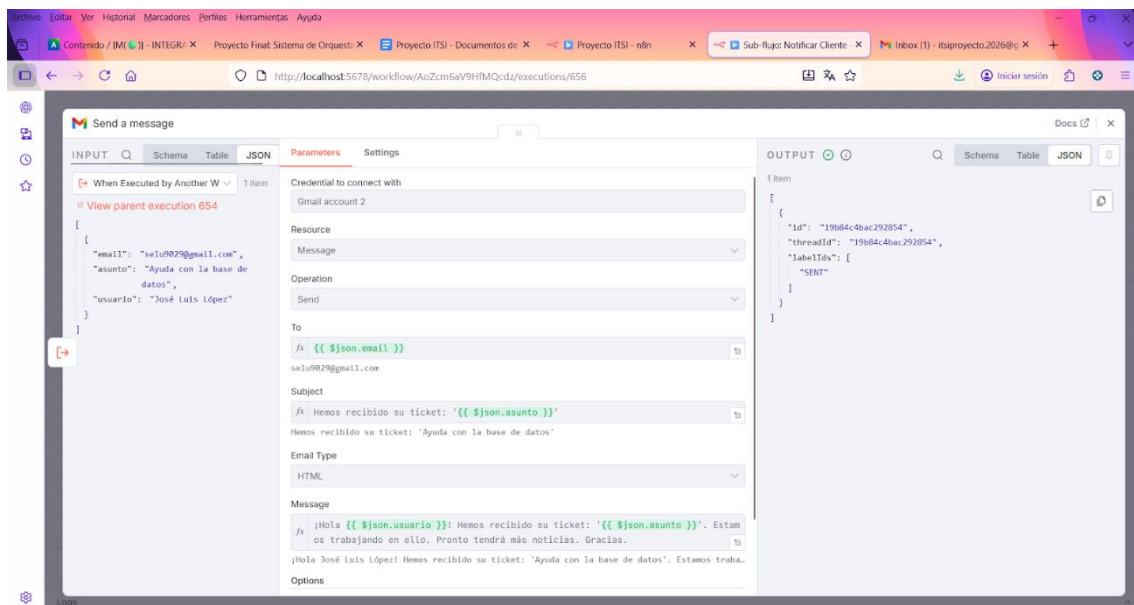


Este flujo secundario recibirá el email del cliente y el resumen del ticket. Utilizará un nodo Send a message para enviar un mensaje de confirmación al cliente

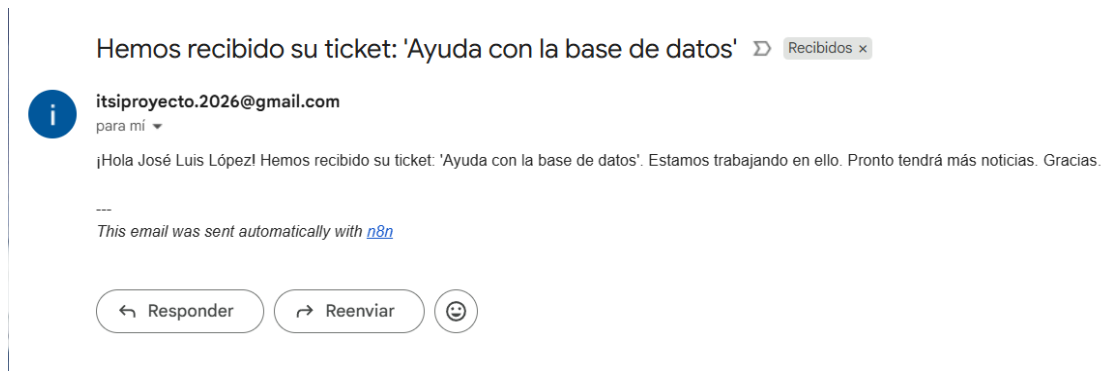
El nodo Execute Workflow Trigger, recibe los datos del cliente



Una vez recibida la información del cliente, se emplea un nodo Send a message, que se encarga de enviar un correo electrónico de confirmación al usuario. El mensaje informa de que el ticket ha sido recibido correctamente y se está trabajando en él.



Este es el correo de respuesta que le llega al cliente.



2. Flujo de Trabajo Secundario: Manejo de Errores

Se ha implementado un flujo de trabajo secundario de manejo de errores global, encargado de capturar y gestionar cualquier fallo que se produzca.

Este flujo permite detectar errores derivados de diferentes situaciones, como por ejemplo:

- Fallos en llamadas a APIs externas (como la API de Gemini).
- Errores de validación de datos.
- Problemas de conectividad con la base de datos.
- Errores en subflujos o nodos internos de n8n.

De esta forma, el sistema no solo detecta el error, sino que lo registra y notifica automáticamente, facilitando la trazabilidad y el diagnóstico posterior.

2.1. Activación del flujo mediante Error Trigger

El flujo de manejo de errores comienza con un nodo Error Trigger, el cual se activa de forma automática cuando cualquier nodo de un flujo principal o subflujo falla durante su ejecución.

Este nodo no requiere datos de entrada manuales, ya que n8n proporciona internamente un objeto execution que contiene información detallada del error ocurrido, incluyendo:

- El mensaje descriptivo del error.
- El flujo de trabajo afectado.
- El enlace directo a la ejecución fallida.

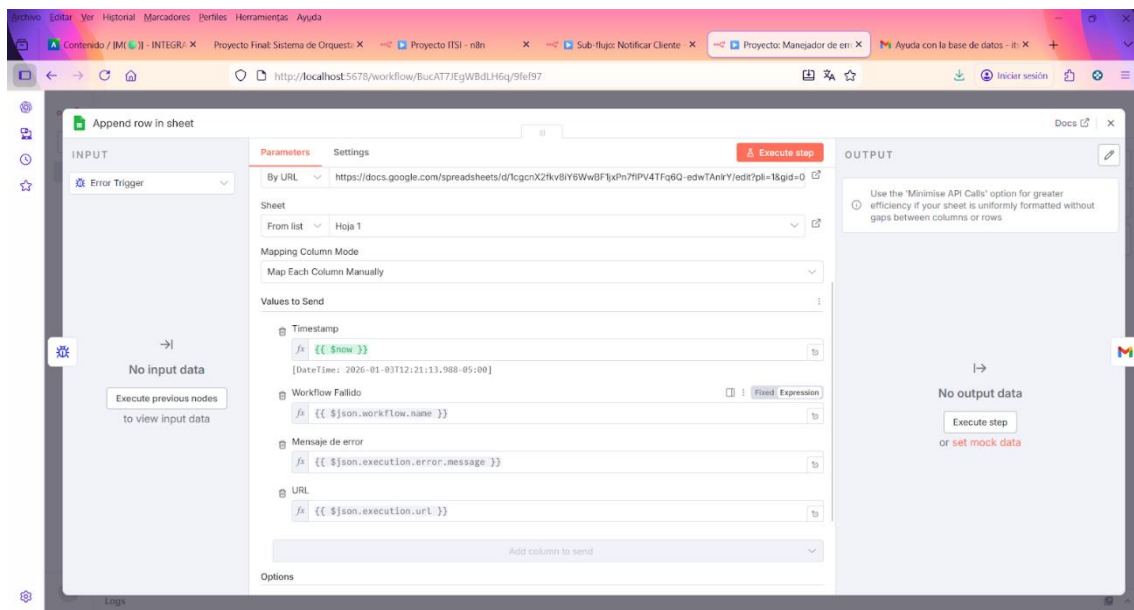
2.2. Registro persistente de errores

Una vez detectado el error, el flujo procede a registrar la información en un Google Sheets.

Para ello, se utiliza el nodo *Append row in sheet*, que añade una nueva fila a la hoja de cálculo cada vez que se produce un fallo.

Los campos registrados son:

- **Timestamp:** Fecha y hora exacta en la que ocurre el error.
- **Workflow Fallido:** Nombre del flujo de trabajo donde se produjo el error.
- **Mensaje de error:** Descripción del error capturado.
- **URL:** Enlace directo a la ejecución fallida en n8n.



Como resultado, el documento se va rellenando con cada error.

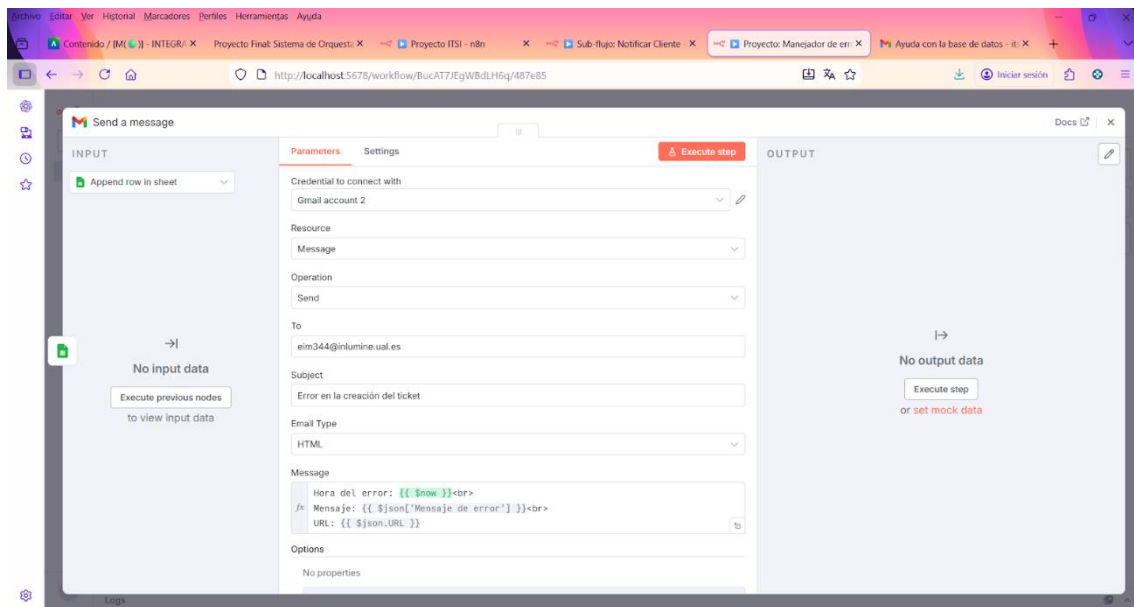
	A	B	C	D	E	F	G	H	I	J	K
1	Timestamp	Workflow Fallido	Mensaje de error	URL							
2	2025-12-26T12:56:22.185-05:00	Proyecto_ITSI	Bad request - please check your parameters	http://localhost:5678/workflow/bWF48OWHKGgpoQpc/execution/551							
3	2025-12-26T12:57:06.115-05:00	Proyecto_ITSI	Bad request - please check your parameters	http://localhost:5678/workflow/bWF48OWHKGgpoQpc/execution/553							
4	2025-12-26T12:58:57.023-05:00	Proyecto_ITSI	Bad request - please check your parameters	http://localhost:5678/workflow/bWF48OWHKGgpoQpc/execution/556							
5	2025-12-26T13:24:25.746-05:00	Example Workflow	Example Error Message	http://localhost:5678/workflow/1/231							
6	2025-12-28T06:35:46.499-05:00	Sub-flujo: Notificar Cliente	Invalid email address (item 0)	http://localhost:5678/workflow/AoZcmBaV9HJHMQcdz/execution/614							
7	2025-12-28T06:36:30.907-05:00	Sub-flujo: Notificar Cliente	Invalid email address (item 0)	http://localhost:5678/workflow/AoZcmBaV9HJHMQcdz/execution/617							
8											
9											

2.3. Notificación automática por correo electrónico

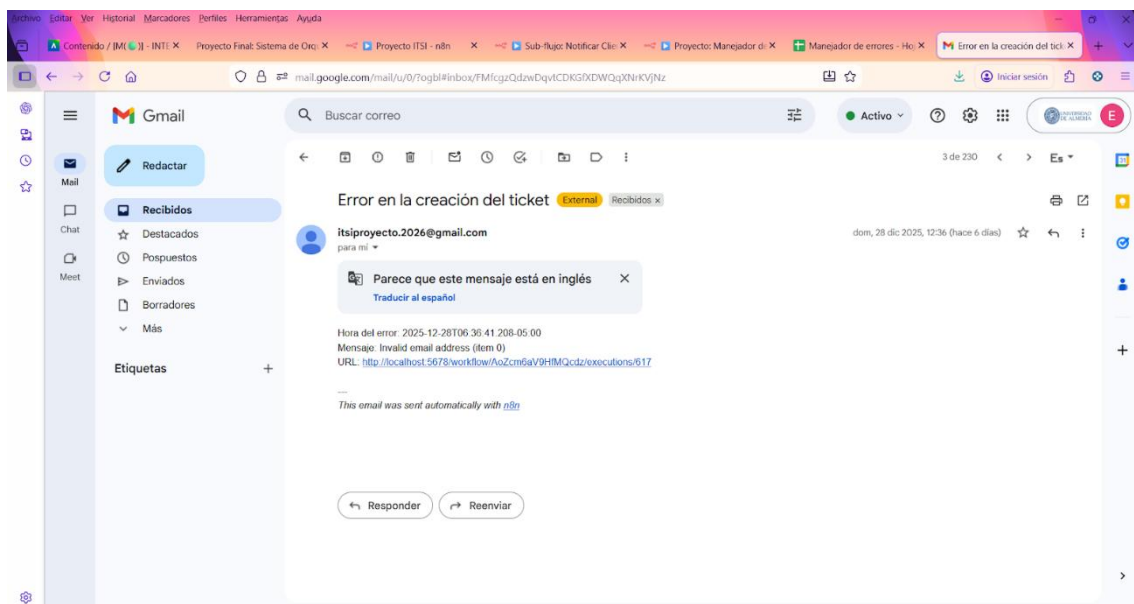
Además del registro en Google Sheet, el flujo de errores envía una notificación automática por correo electrónico al administrador del sistema cada vez que se produce un fallo.

Para ello, se utiliza el nodo *Send a message* de Gmail, configurado para enviar un correo en formato HTML que incluye:

- La hora del error.
- El mensaje descriptivo del fallo.
- El enlace directo a la ejecución en n8n.

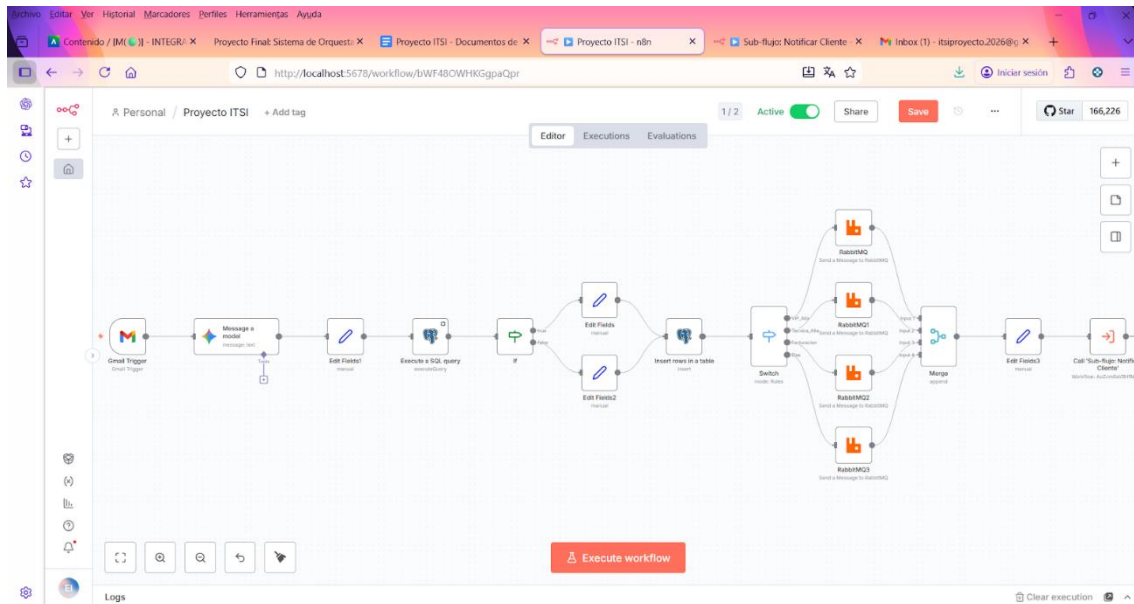


Como resultado, el correo que recibe el administrador es:

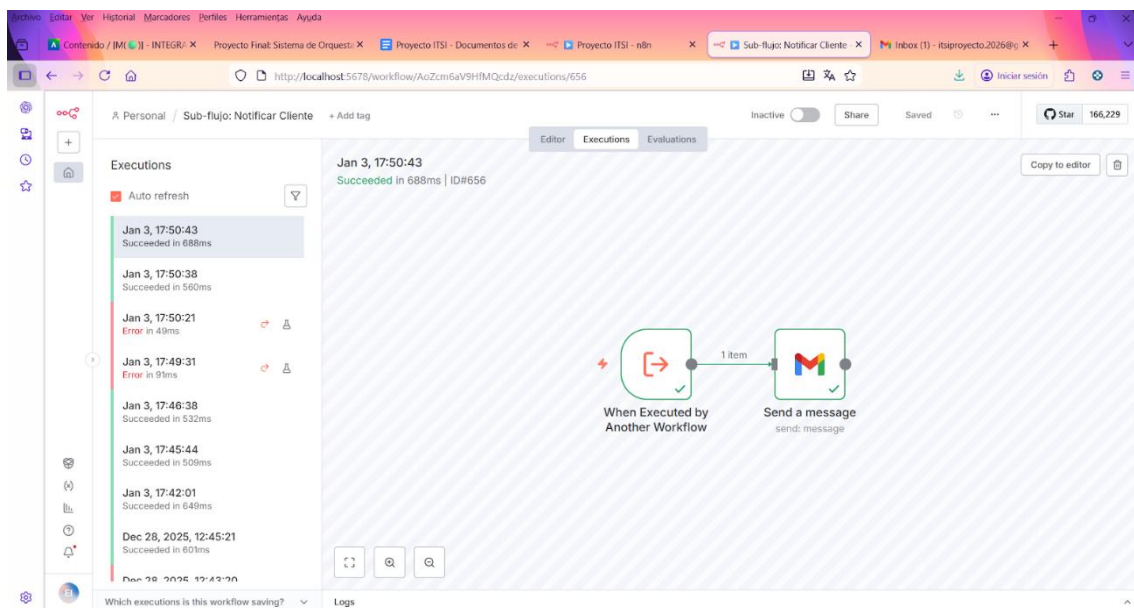


3. Flujos resultantes

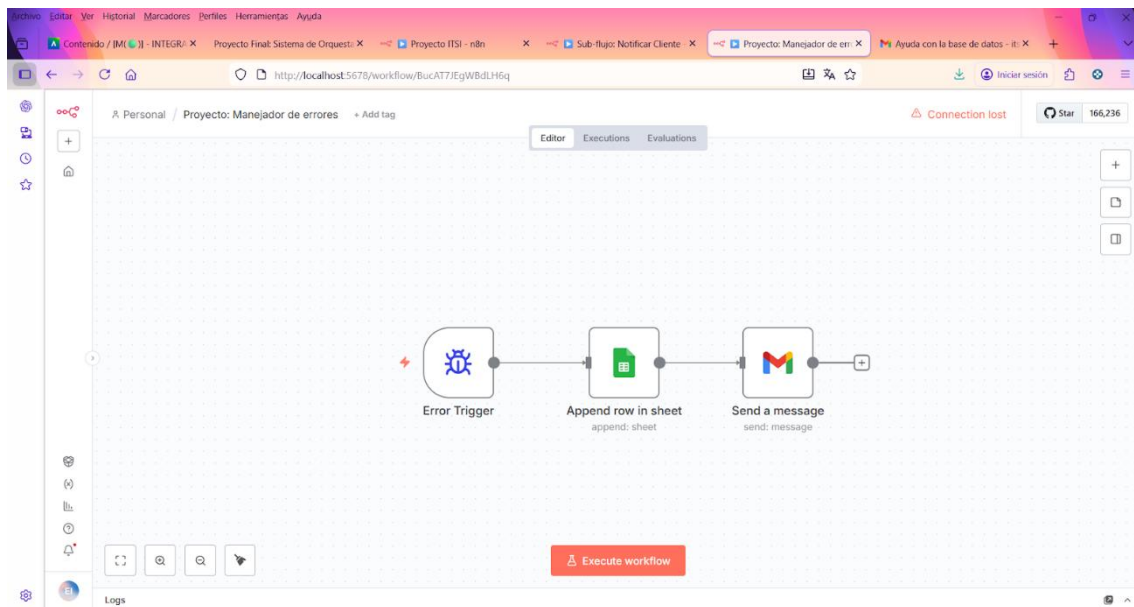
Flujo principal:



Sub-flujo: Notificar Cliente:



Manejador de errores:



4. Establecer las variables de entorno

Para el correcto funcionamiento del proyecto se van a usar variables de entorno, con el objetivo de centralizar la configuración (credenciales, puertos, nombre de servicio...). De esta forma, el proyecto queda portable y puede desplegarse en cualquier equipo simplemente rellenando un fichero `.env`.

4.1. Variables de entorno utilizadas

Las variables que se han utilizado para levantar la pila y para que n8n pueda conectarse a los servicios externos (BBDD, colas y APIs), son las siguientes.

4.1.1. Base de datos – Postgre SQL

Estas variables permiten crear la base de datos y autenticar las conexiones desde n8n al ejecutar los nodos PostgreSQL.

- **POSTGRES_USER:** usuario de PostgreSQL.
- **POSTGRES_PASSWORD:** contraseña del usuario.
- **POSTGRES_DB:** nombre de la base de datos que se inicializa al arrancar el contenedor.

- **POSTGRES_HOST:** el nombre del servicio en Docker.

```
POSTGRES_USER=user
POSTGRES_PASSWORD=password
POSTGRES_DB=taskdb
POSTGRES_HOST=db
```

4.1.2. Cola de mensaje – RabbitMQ

Credenciales para el broker de mensajería:

- **RABBITMQ_USER:** usuario inicial de RabbitMQ.
- **RABBITMQ_PASSWORD:** contraseña del usuario inicial.
- **RABBITMQ_URL:** URL que utilizará n8n (y/o cualquier consumidor) para publicar mensajes en las colas.
- **RABBITMQ_HOST:** el nombre del servicio en Docker.

```
RABBITMQ_HOST=mq
RABBITMQ_USER=guest
RABBITMQ_PASSWORD=guest
RABBITMQ_URL=amqp://guest:guest@mq:5672/%2F
```

4.1.3. N8N

Estas variables permiten configurar el despliegue de n8n en local (puerto y host dentro de la red Docker) y controlar parámetros de seguridad relacionados con la gestión de sesión, como el uso de cookies seguras cuando se trabaja bajo HTTPS.

- **N8N_PORT:** puerto donde se expone n8n en local (por defecto 5678).
- **N8N_HOST:** el nombre del servicio en Docker.
- **N8N_SECURE_COOKIE:** controla si las cookies de sesión de n8n deben enviarse sólo a través de conexiones HTTPS seguras.

```
N8N_SECURE_COOKIE=false
N8N_HOST=localhost
N8N_PORT=5678
```

4.1.4. Cuenta de correo – Notificaciones

Estas variables permiten identificar la cuenta de correo desde la que el sistema enviará notificaciones al producirse errores. **GMAIL_EMAIL**: dirección de correo utilizada por el sistema al manejar los errores.

```
GMAIL_EMAIL=eim344@inlumine.ual.es
```

4.1.5. Registro de errores – Google Sheet

Estas variables permiten indicar a n8n en qué documento y hoja de Google Sheets se registran los errores cuando se dispare el flujo global Error Trigger, dejando trazabilidad de las incidencias generadas durante la ejecución.

- **GOOGLE_SHEETS_ID (ID del documento de Google Sheets)**: permite registrar errores cuando se dispare el flujo global de Error Trigger.
- **GOOGLE_SHEETS_NAME (Nombre de la hoja de Google Sheets)**: permite registrar errores en una hoja definida.

```
GOOGLE_SHEETS_ID=1cgcnX2fkv8iY6WwBF1jxPn7f1PV4TFq6Q-edwTAnlrY  
GOOGLE_SHEET_NAME=Hoja_1
```

4.2. Ejemplo de .env.example

El fichero .env.example actúa como plantilla.

```
# =====  
# DATABASE CONFIGURATION  
# =====  
POSTGRES_USER=your_db_user  
POSTGRES_PASSWORD=your_db_password  
POSTGRES_DB=your_database_name  
POSTGRES_HOST=db  
  
# =====  
# RABBITMQ CONFIGURATION  
# =====  
RABBITMQ_HOST=mq  
RABBITMQ_USER=guest  
RABBITMQ_PASSWORD=guest  
RABBITMQ_URL=amqp://guest:guest@mq:5672/%2F
```

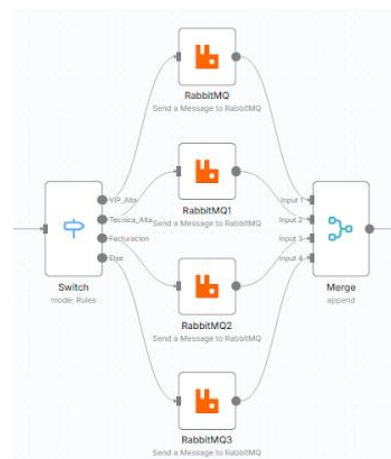
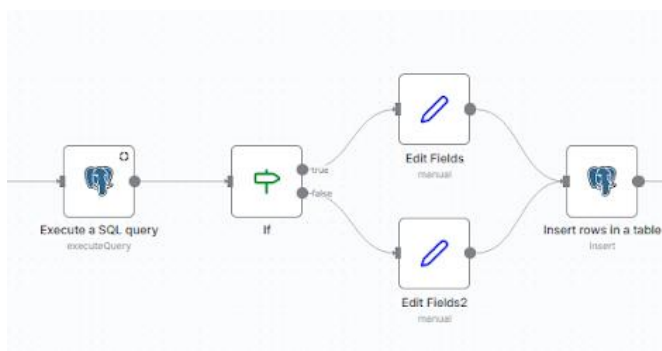
```
# =====
# N8N CONFIGURATION
# =====
N8N_SECURE_COOKIE=false
N8N_HOST=localhost
N8N_PORT=5678

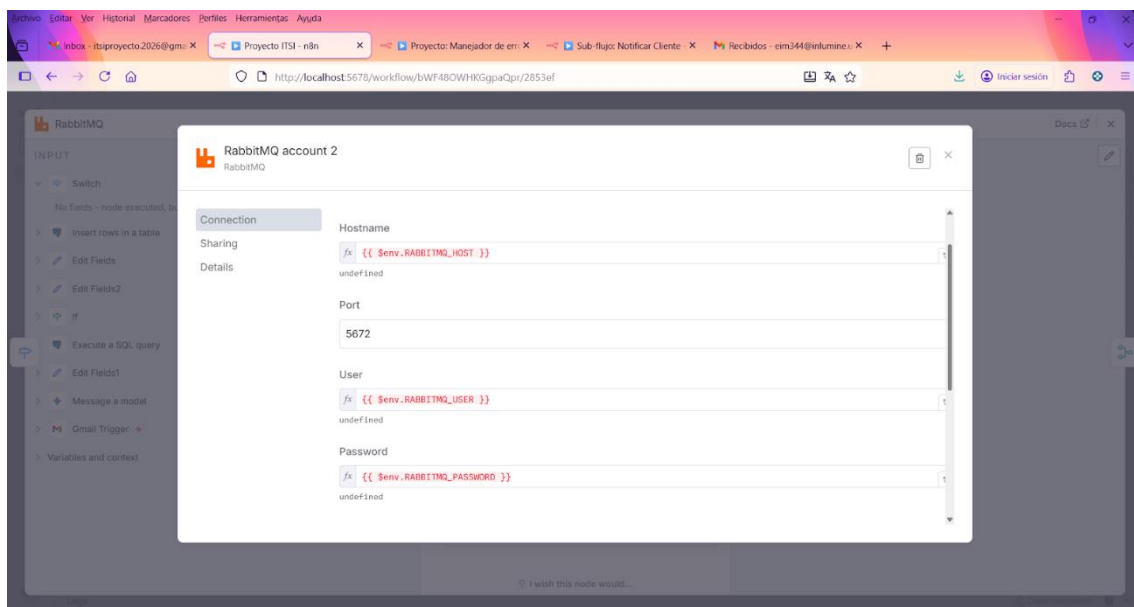
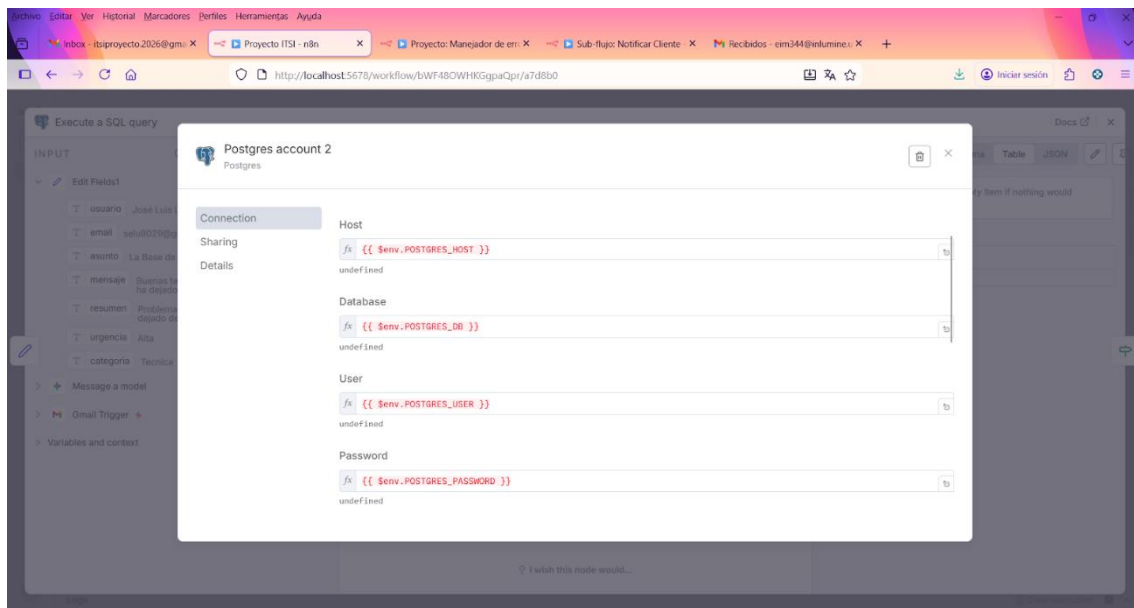
# =====
# GOOGLE SHEETS CONFIGURATION
# =====
GOOGLE_SHEETS_ID=your_google_sheets_id_here
GOOGLE_SHEET_NAME=your_sheet_name

# =====
# GMAIL CONFIGURATION
# =====
GMAIL_EMAIL=your_email@example.com
```

4.3. Establecer las variables de entorno en n8n.

En el proyecto principal se tienen que modificar las credenciales de PostgreSQL y de RabbitMQ.





Y del manejador de errores los nodos de Google Sheets y de Gmail:

